01 / 11 · INTRODUCTION

AI Agents Components

How they think, plan, and act —
the building blocks explained simply.

Tool Calling | Function Calling | Planner | Reflection | Multi-Agent | Memory | ReAct Loop

What is an AI Agent? CORE

An LLM that doesn't just respond — it takes actions in a loop until a goal is reached. Unlike a chatbot, it decides what to do next.


```
# An agent is an LLM in a while loop
while not done:
    action = llm.decide(context)
    result = execute(action)
    context.append(result)
```

One Kit to Clear Your AI Interview KIT

After seeing how AI interviews are evolving, I created an **AI Interview Kit.** It covers the areas companies actually test:

- AI fundamentals
- AI agents
- Transformers (in depth)
- Memory systems
- LLM architectures
- Databases
- Prompt engineering
- Backend for AI systems
- Fine-tuning
- Deployment strategies
- RAG systems
- System design for GenAI

Everything structured to help you prepare for modern AI interviews.

↓ Check the caption to enroll!

Tool Calling

ACTIONS

The LLM can invoke external tools — web search, calculators, databases — and receive the results back into its context.

WEB SEARCH
Look up real-time info

CODE RUNNER
Execute calculations

DATABASE
Query SQL & write

```
# Define tools for the LLM
tools = [
  {"name": "search", "desc": "Search web"},
  {"name": "run_code", "desc": "Eval code"}
]
response = llm.call(prompt, tools=tools)
# {"tool": "search", "args": {"q": "AI"}}
```

Function Calling

STRUCTURED

The LLM outputs structured JSON describing which function to call and with what args. Your code executes it — the result feeds back in.

LLM OUTPUT

```
{"func": "email",
"args": {"to": "a@co"}}
```

YOUR CODE RUNS IT

```
email(
  to="a@co"
)
```

```python
def send_email(to, subject):
    # Your actual business logic
    email.send(to=to, subject=subject)
    return {"status": "sent"}

# Result feeds back to LLM
```

Planner Agent

STRATEGY

Breaks a complex goal into ordered sub-tasks, then executes them one by one. Great for multi-step goals: research → write → review → publish.


```
plan = llm.plan("Write AI blog")
# → ["research", "write", "publish"]

for step in plan:
    result = execute(step)
    context.add(result)
```

Reflection Agent

QUALITY

The agent critiques its own output. A second LLM call acts as a critic — it reviews and scores the draft, triggering a revision loop.

GENERATOR LLM

Produces the first draft of the output.

CRITIC LLM

Reviews, scores, and suggests improvements.

↺ loop repeats until quality threshold is met

```
draft = generator.run(task)
feedback = critic.review(draft)

while feedback.score < threshold:
    draft = generator.revise(draft, feedback)
    feedback = critic.review(draft)
```

Agent-to-Agent

MULTI-AGENT

A supervisor agent breaks a task and delegates to specialist sub-agents. Each agent has a focused role; results flow back up to the supervisor.

RESEARCH AGENT

Gathers facts, searches web.

WRITING AGENT

Drafts content easily.

REVIEW AGENT

Quality checks output.

```
sup.send(research_agent, "Find papers")
facts = research_agent.run()

sup.send(writing_agent, task=facts)
draft = writing_agent.run()
```

Memory Types

PERSISTENCE

Agents use different memory types to persist knowledge across steps and sessions. Choose the right type for your usecase.

SHORT-TERM

In prompt window. Fast but lost when session ends.

LONG-TERM

Vector DB, SQL. Persists across sessions.

EPISODIC

Past experiences: "last time it resulted in Y".

SEMANTIC (RAG)

Facts stored as embeddings, similarity search.

```
memory.store(key="notes", val=summary)
ctx = memory.retrieve(query="meeting")
```

ReAct Loop

REASON + ACT

The most common agent pattern. The LLM alternates between Thought, Action, and Observation until it reaches the final answer.

Thought LLM reasons → **Action** Calls a tool → **Observation** Gets result back

```
Thought: I need Paris weather.
Action:  search("Paris weather")
Observe: "Partly cloudy, 18°C"
Thought: I have the answer.
Answer:  "Paris is 18°C."
```

The Agent Stack

ALL TOGETHER

Every agent is a combination of these components. Mix and match — most real-world agents use 3–5 of these together.

01
LLM Core

02
Tool Calling

03
Function Call

04
Planner

05
Reflection

06
Multi-Agent

07
Memory Types

08
ReAct Loop

∞

Your Agent

Combine what fits your use case